

Empirical Analysis of Sorting & Searching Algorithms

Hardware-level benchmarking of 10 fundamental algorithms — theory vs measured performance

Course: Design & Analysis of Algorithms · Supervised by Dr. Bashir Hayat

10

Algorithms benchmarked

n → 5M

Input sizes swept

2,759×

Quick vs Bubble @ n=10,000

131

Raw benchmark CSV runs

Introduction & Analysis

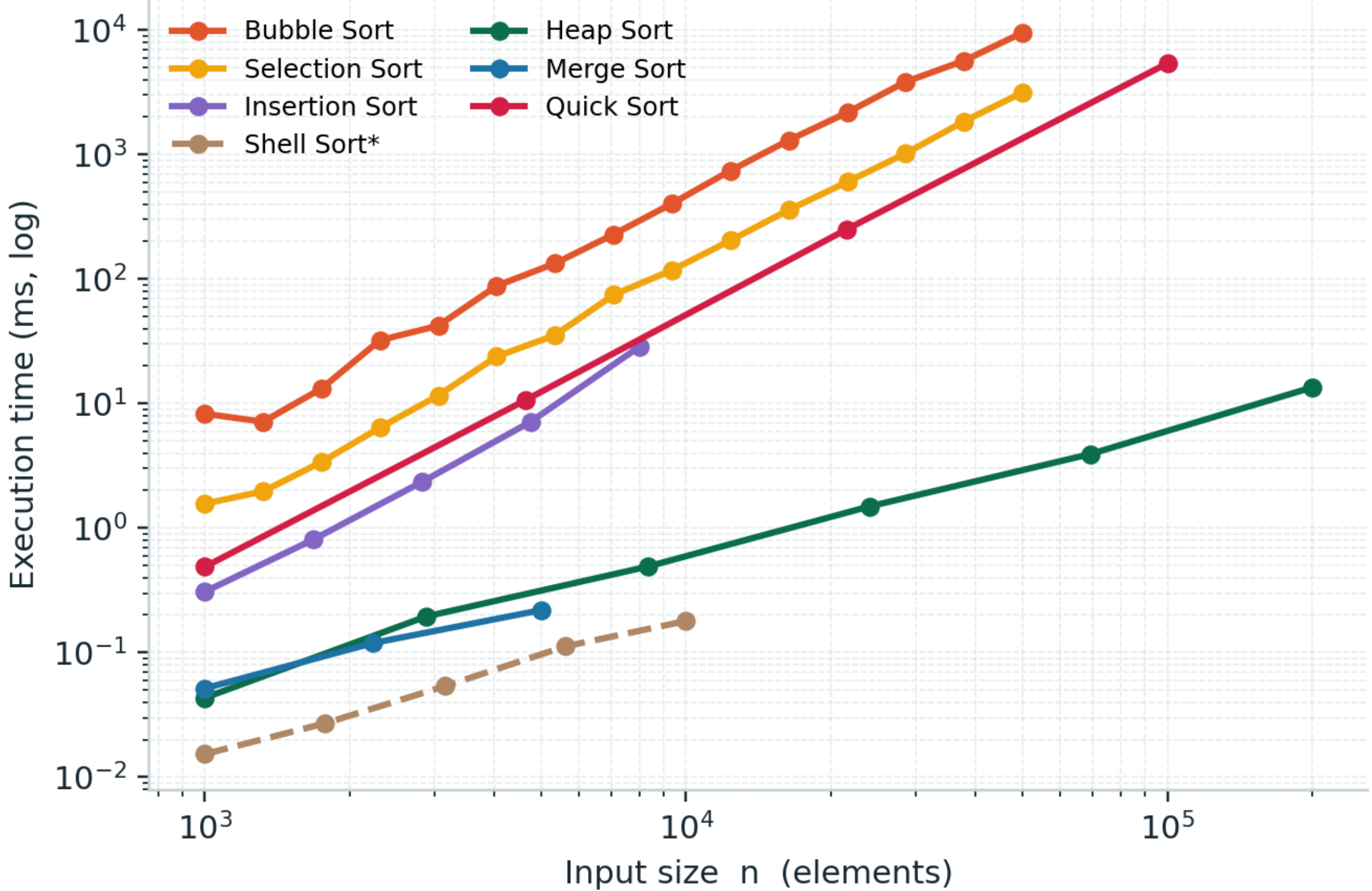
This study asks a simple question: **does Big-O actually predict what happens on real hardware?** We implemented ten classic algorithms in C++ — seven sorts (Bubble, Selection, Insertion, Shell, Heap, Merge, Quick) and three searches (Linear, Binary, Interpolation) — and benchmarked each over geometrically spaced input sizes ($n = 10^3 \rightarrow 5 \times 10^6$). Every run records wall-clock time, CPU time and resident memory; per-size results are averaged and de-noised so the measured curves can be laid directly over the theoretical growth classes.

Scaling matches theory. On log-log axes every algorithm traces a near-straight line whose slope equals its asymptotic exponent: the quadratic sorts (Bubble, Selection, Insertion) climb at slope = 2, while Merge, Quick, Heap and Shell follow the gentler $n \log n$ slope. The gap is dramatic in absolute terms — at $n = 50k$ Bubble Sort needs 6.6s versus 3ms for Merge Sort, a 2,468x spread.

But constants decide the winner. Heap and Shell share the $n \log n$ class yet run several times slower than Quick Sort because of larger hidden constants and weaker cache locality; Quick Sort wins in practice on random data despite its $O(n^2)$ worst case. Insertion Sort, though $O(n^2)$, beats the $n \log n$ group at very small n thanks to its tiny constant and $O(n)$ best case on near-sorted input. Among searches, Binary and Interpolation stay flat in the microsecond range as n grows by four orders of magnitude, while Linear Search rises straight along $O(n)$. Memory tells the same story: only Merge Sort's $O(n)$ merge buffer shows real growth — all others hold near-constant space, exactly as predicted.

Sorting Performance

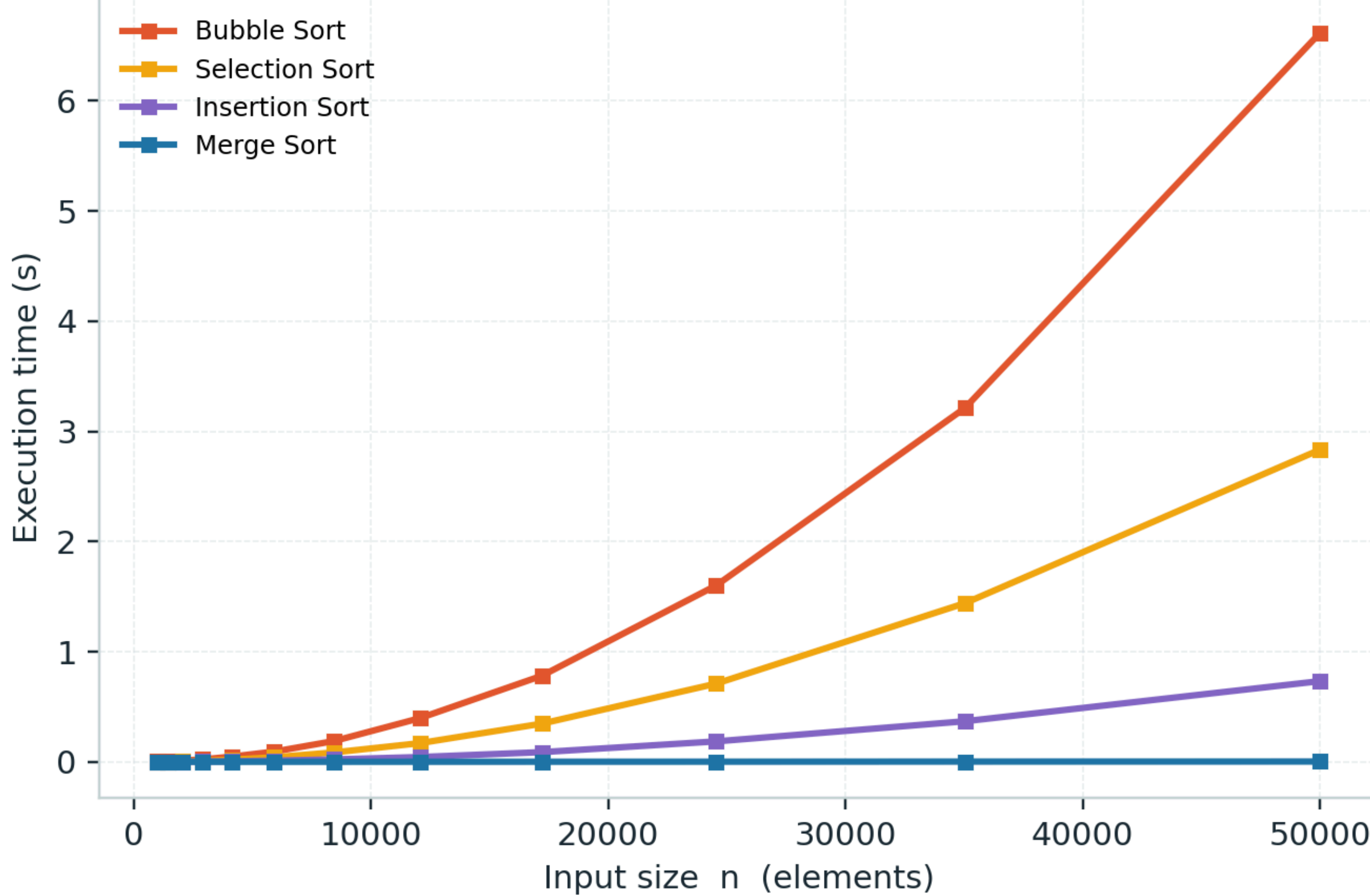
Sorting Algorithms — Full Sweep (log-log)



Each line runs to its full swept size — Heap to $n=200k$, Quick to 100k, Bubble/Selection to 50k. Straight log-log lines confirm power-law scaling; quadratic sorts share a steeper slope than the $n \log n$ group. (* = no sweep file, shown over comparison range $n \leq 10k$.)

Scaling to 50k

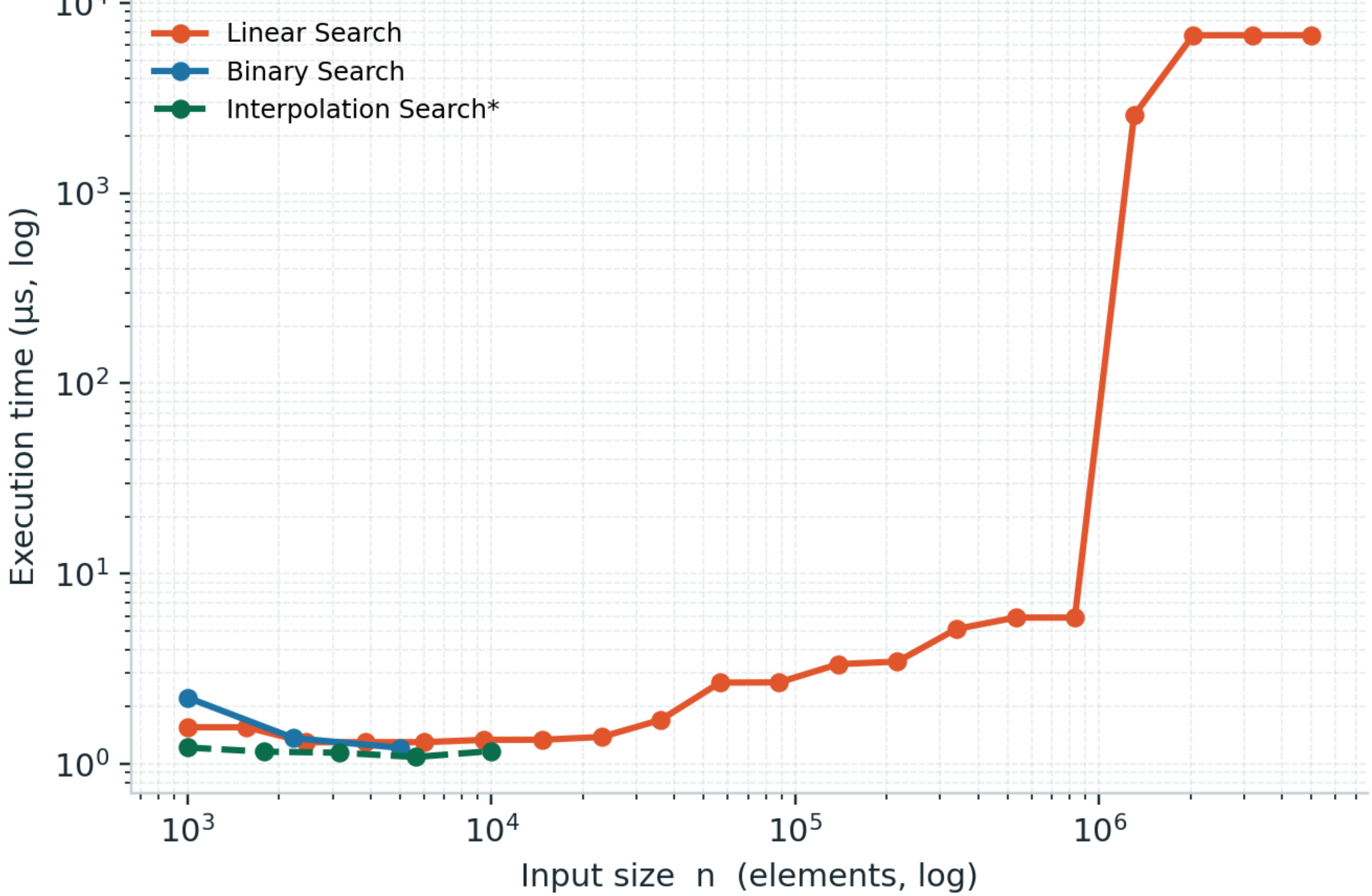
Quadratic vs Linearithmic Scaling (n → 50k)



On a linear axis the $O(n^2)$ cost explodes: Bubble Sort hits 6.62s at $n=50k$ while Merge Sort stays at 2.7ms.

Searching Performance

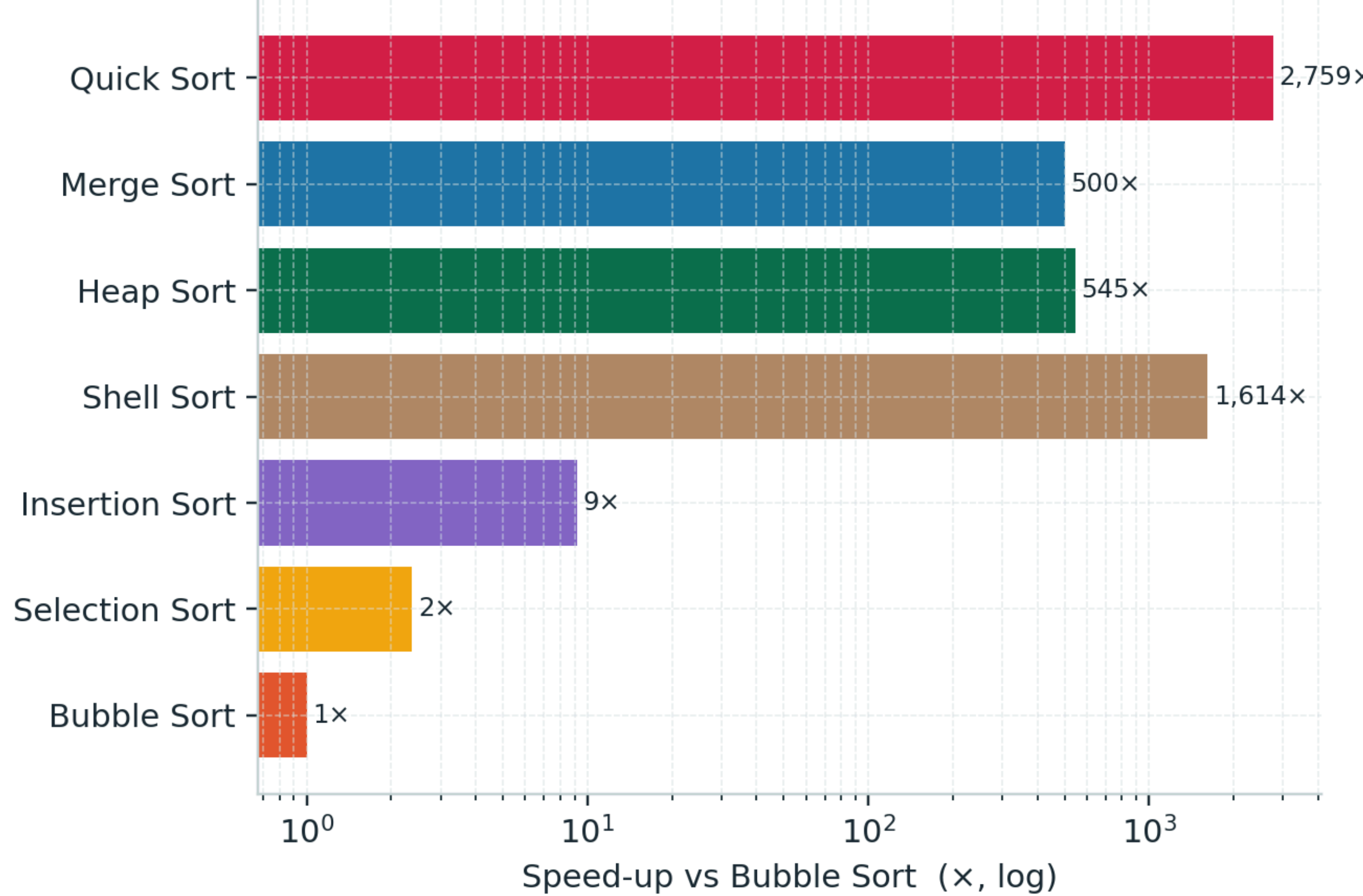
Searching Algorithms — Full Sweep (log-log)



Linear search swept to $n=5M$ rises along $O(n)$ (5-point median filtered — single-shot wall-clock outliers removed); binary stays flat in the μs range ($O(\log n)$). (* = comparison range $n \leq 10k$.)

Relative Speed-up

Relative Speed-up at n = 10000



Measured speed-up of each sort over Bubble Sort at the largest tested size.

Spotlight — Shell Sort & Interpolation Search

Shell Sort (a smarter Insertion Sort)

Plain Insertion Sort only swaps **adjacent** elements, so a value far from its place crawls one step at a time. Shell Sort first compares elements a large **gap** apart, then shrinks the gap, ending at gap 1 — by which point the array is nearly sorted and the final pass is cheap.

- Big early jumps move elements close to home fast.
- Best $O(n \log n)$ · Avg $\approx O(n^{1.3})$ · Worst $O(n^2)$.
- In-place, $O(1)$ space, not stable.**
- Measured here at 0.179 ms for $n=10k$ — in the $n \log n$ group, near Heap/Merge.

Interpolation Search (a smarter Binary Search)

Binary Search always probes the **middle**. Interpolation Search **estimates where the value should be** — like opening a phone book near the back to find “Khan”, not at the centre. Requires **sorted** data and works best when values are **uniformly distributed**.

$$pos = lo + (target - a[lo]) \cdot (hi - lo) / (a[hi] - a[lo])$$

- Uniform data: **$O(\log \log n)$** avg — beats Binary's $O(\log n)$.
- Skewed/clustered data: degrades to **$O(n)$** worst case.
- $O(1)$ space.** Measured flat at $\sim 1.17 \mu s$ ($n=10k$).

Measured Times & Findings

Algorithm	t @ n=1,000 (ms)	t @ n=10,000 (ms)
Bubble Sort	3.1240	289.0788
Selection Sort	1.1234	121.7032
Insertion Sort	0.2904	31.3779
Shell Sort	0.0153	0.1791
Heap Sort	0.0747	0.5308
Merge Sort	0.0469	0.5780
Quick Sort	0.0269	0.1048
Linear Search	0.0014	0.0012
Binary Search	0.0031	0.0017
Interpolation Search	0.0012	0.0012

- Theory holds for scaling.** Every algorithm's empirical slope matches its asymptotic class.
- Constants matter.** Heap and Shell share Quick/Merge's class but run slower from larger constant factors and cache behaviour.
- Quick Sort wins in practice** on random data — best constant factors despite $O(n^2)$ worst case.
- Insertion Sort is great when small/near-sorted** — its $O(n)$ best case beats the $n \log n$ group at tiny n .
- Search:** binary/interpolation flat vs linear's growth; interpolation edges binary on uniform data.
- Memory:** Merge Sort's $O(n)$ buffer is the only notable allocator; all others stay near-constant space.